Infraestrutura de Software
Relatório Implementação 4

Aluno: Arthur Rodrigues de Andrade
Turma: 3B

1. Objetivo

Este trabalho implementa, em C, funções de sincronização para uma estação de metrô com um carro por vez e vários passageiros, cada um controlado por uma thread. O carro só sai quando todos os passageiros que embarcaram estão sentados e o carro está cheio ou não há mais ninguém esperando. São entregues **sync_car.h** e **sync_car.c**, com a struct **station** e as funções **station_init, station_load_car, station_wait_for_car e station_on_board**, usando apenas um mutex e variáveis de condição, sem semáforos e sem espera ocupada.

2. Checklist de Requisitos

Requisito do enunciado	Status	Como testar / evidência
Makefile (compila e limpa)	OK	`make / make clean`
Embarque simultâneo e sem busy-wait	OK	`./sync_car`
Só funções pthread permitidas, um único lock, sem semáforos	OK	No código
Struct station e as 4 funções em C	OK	`make`
Teste no runner	OK	`./sync_car` termina com `Looks good!`
Casos limite e Caso Inválido	OK	`evidencias.log`
README	OK	`README.md` no repositório

3. Como Reproduzir

O projeto utiliza um **Makefile** para facilitar o processo de compilação e a limpeza dos arquivos gerados. Para compilar o projeto, basta executar:

```
make
```

Obs: Durante a compilação aparece um aviso de parâmetro não usado (`foo`) em `sync_car-runner.c`. Ele vem do código do professor, que não foi alterado, e não afeta o funcionamento.

Após a compilação, o programa é executado sem argumentos:

```
./sync_car
```

O programa simula carros com número aleatório de assentos passando pela estação até que os 100 passageiros embarquem. A execução está correta quando a saída termina com `Looks good!`.

Para detectar falhas intermitentes, o teste pode ser repetido várias vezes (o esperado é 100 `Looks good!`):

```
for i in $(seq 100); do ./sync_car | tail -1; done | sort | uniq -c
```

Por fim, para remover o executável e os arquivos gerados, utiliza-se:

```
make clean
```

4. Arquitetura (resumida)

Arquivos e responsabilidades:

`sync_car.c` → implementa `station_init`, `station_wait_for_car`, `station_on_board` e `station_load_car`.

`sync_car.h`→ define a struct `station` e declara as 4 funções.

`sync_car-runner.c`→ teste do professor, sem alterações.

Makefile → compila e limpa

Decisões técnicas:

1. `while` no lugar de `if` → Motivo: quando o carro chega, todos os passageiros acordam, mas nem todos têm lugar → Consequência: quem ficou sem lugar confere de novo e volta a esperar.

2. Contador `embarcando` → Motivo: o carro não pode sair enquanto tem passageiro que ainda não sentou → Consequência: cada passageiro avisa ao sentar, e o carro só parte quando todos avisaram.

3.**`station_load_car` retorna na hora se o carro tem 0 assentos ou se ninguém está esperand**o → Motivo: o programa de teste exige que o carro não fique parado nesses casos → Consequência: o carro parte vazio, sem travar o programa.

5. Estratégias e Diário de Desenvolvimento

5.1 Estratégias da semana

Estratégia	Nome curto	Contexto	Motivo da troca (se houve)
S1	Um mutex com duas condições de espera	A estação é um ponto de encontro entre o carro e os passageiros, com duas esperas: os passageiros esperam um carro com lugar livre, e o carro espera todos os passageiros liberados sentarem. Tudo é protegido por um único mutex, como o enunciado exige. Implementei uma função por vez, compilando a cada etapa, e só testei o conjunto com o programa do professor no final.	Não houve.

5.2 Diário de Tentativas

#	Estrat.	O que tentei	Resultado	Hipótese/ Causa (se falhou)	Quando	Evidência
1	S1	Estrutura inicial do projeto (GitHub, .gitignore e aquivos)	OK		29/10	github (commits `735edaf` e `ef3f6ed`)

2	S1	Implementar struct station em `sync_car.h`	OK		29/10	github (commit `be2cfd0`)
3	S1	Implementar `station_init` (deixa a estação pronta para uso)	OK		29/10	github (commit `c563e5f`)
4	S1	Implementar `station_wait_for_car` (o passageiro espera um carro com lugar livre e reserva o assento)	OK		30/10	github (commit `8211ec6`)
5	S1	Implementar `station_on_board` (o passageiro avisa que já sentou)	OK		30/10	github (commit `523038f`)
6	S1	Implementar `station_load_car` (o carro espera todos sentarem e só então parte)	OK		30/10	github (commit `29360a6`)
7	S1	Adicionar o programa de teste do professor e o Makefile, e compilar com `make`	OK	O aviso (`unused parameter 'foo'`) está no código do professor, que não foi alterado	03/11	evidencias.log e github (commits `a77b622` e `5874631`)
8	S1	Testar com o runner: uma primeira execução e depois 20 execuções seguidas	OK		03/11	evidencias.log e github (commit `2230878`)
9	S1	Ignorar o executável no `.gitignore`, versionar o log	OK		03/11	github (commits `2025743` e `f1a0914`)

		e escrever o README				
10	S1	Testar o caso inválido (carro com número negativo de assentos) e caso limite (carro com 0 assentos e carro com exatamente o número de passageiros esperando) e fazer a validação final (`make clean`, `make`, 40 e 100 execuções seguidas)	OK		04/11	evidencias.log e github (commit `2bd618c`)

6. Evidências

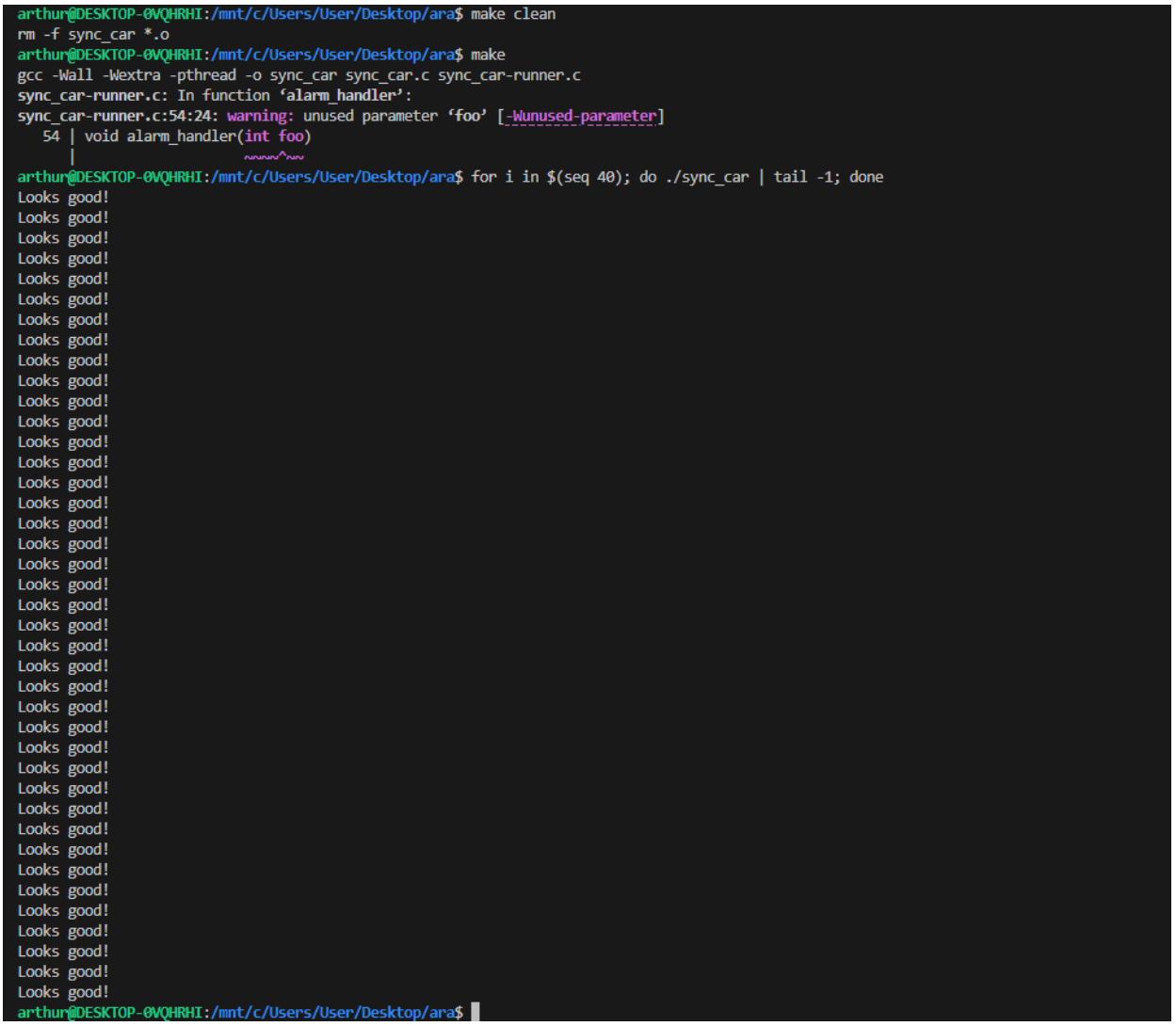
6.1 Log automático (faz o trabalho pesado por você)

No início de cada sessão de trabalho, foi executado:

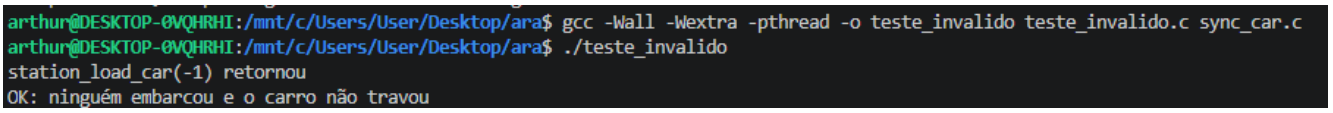
script -a evidencias.log

Isso gravou automaticamente todos os comandos digitados e as saídas do terminal em um único arquivo. O evidencias.log está sendo enviado dentro do .tar de entrega

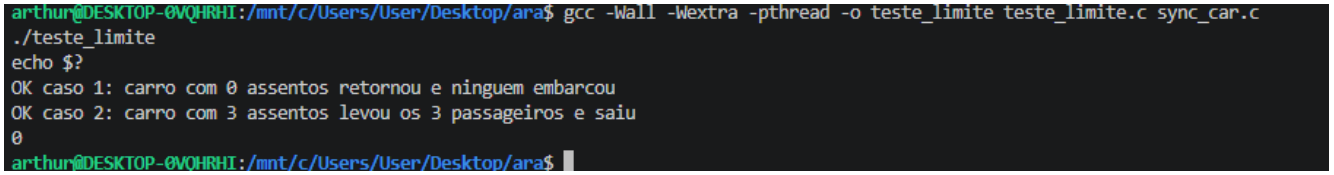
6.2 Prints



• **Validação final:** o projeto foi compilado do zero com `make clean e make`, e o `./sync_car` terminou com `Looks good!`. Em 40 execuções seguidas, todas terminaram com `Looks good!`. A evidência de todos os testes está no `evidencias.log`.



• **Caso inválido:** foi testado um carro chegando com número negativo de assentos (-1), com um passageiro esperando. A função retornou sem travar e o passageiro não embarcou.



• **Casos limite:** foram feitos dois testes. No primeiro, um carro chegou com 0 assentos com passageiros

esperando: a função retornou na hora e ninguém embarcou. No segundo, um carro com 3 assentos chegou com 3 passageiros esperando: os 3 embarcaram, o carro só saiu depois de todos avisarem que sentaram.

7. Uso de IA

Onde usei IA (2–3 linhas): Usei a IA como apoio durante o trabalho, principalmente para entender o problema de sincronização e esclarecer dúvidas durante a implementação, e principalmente, me ajudou a entender as bibliotecas Pthreads, como `pthread_mutex_t` e `pthread_cond_t`, além de orientar na montagem do README e dos programas de teste de caso limite e caso inválido.

Prompts principais (liste, sem história):

- “Me explique o problema de sincronização do sync_car e como funciona sua lógica.”
- “Me ajude a entender as bibliotecas Pthreads, principalmente pthread_mutex_t, pthread_cond_t e suas variações.”
- “Me ajude na montagem do README e na criação de testes para casos limite e entradas inválidas.”

O que validei manualmente e como:

Compilei com `make` a cada etapa e li os avisos do compilador. Rodei o programa do professor uma vez, 20 vezes, 40 vezes e 100 vezes seguidas, e conferi que todas terminaram com `Looks good!`. Rodei os testes de caso limite e de caso inválido e conferi as saídas.

8. Reflexão Final

Reflexão final:

O maior aprendizado foi entender como threads se coordenam sem gastar processador: a thread dorme em uma condição e só volta quando alguém a acorda. Aprendi a importância de usar o mutex e as variáveis de condição para evitar problemas durante o embarque e por que a espera precisa de um while e não de um if, já que várias threads acordam juntas e nem todas têm lugar. Também entendi por que o carro precisa de um contador de passageiros que ainda não sentaram, para não partir cedo. Os testes de caso limite e caso inválido me fizeram ver o programa por um ângulo diferente do teste do runner do professor.

9. Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	Sim
Rodei os testes e evidencias.log foi gerado	Sim
Tenho prints obrigatórios	Sim
Testei pelo menos um caso limite e um caso inválido	Sim
Preenchi a seção de uso de IA	Sim
Revisei o relatório e removi frases genéricas / vazias	Sim

10. Se eu tivesse mais 2 horas

Se tivesse mais 2 horas, usaria esse tempo para testar mais situações, principalmente casos limite e entradas inválidas, para verificar como o programa se comporta. Também rodaria o runner que foi passado pelo professor em umas 500 execuções seguidas para procurar possíveis falhas. Além disso, revisaria o código para conferir se todas as situações estão sendo tratadas corretamente e se existem avisos desnecessários durante a compilação.